The Journal of Open Source Software

Bensemble: A Modular Python Library for Bayesian Deep Learning and Neural Network Ensembling

Dmitrii Vasilenko [1], **Muhammadsharif Nabiev**[1], **Fedor Sobolevsky**[1], **Vadim Kasyuk**[1], **and Oleg Bakhteev**[2]

1 Moscow Institute of Physics and Technology **2** Skolkovo Institute of Science and Technology

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Matt Graham

Reviewers:

- @phanicode
- @TobyB1702
- @Shakti-95

Submitted: 23 July 2026
Published: unpublished

Summary

Modern deep learning models are typically trained to produce a single point prediction, with no indication of how much to trust that prediction. In many research and decision-making settings — medical imaging, autonomous systems, scientific measurement — knowing how uncertain a model is matters as much as the prediction itself. `bensemble` is a Python library, built directly on top of PyTorch, that provides a unified interface for quantifying and using this uncertainty. It brings together several distinct families of methods: explicit ensembling (Deep Ensembles (Lakshminarayanan et al., 2017)), implicit/stochastic ensembling (Monte Carlo Dropout (Gal & Ghahramani, 2016), variational Bayesian layers optimized via the standard evidence lower bound or its Rényi-divergence generalization (Kingma et al., 2015; Li & Turner, 2016)), post-hoc posterior approximations (Laplace approximation with Kronecker-factored curvature (Ritter et al., 2018), Probabilistic Backpropagation (Hernández-Lobato & Adams, 2015)), and ensemble/architecture search (Neural Ensemble Search (Zaidi et al., 2021) via random search, regularized evolution, and a discrete, pool-based sampler inspired by Neural Ensemble Search via Bayesian Sampling (Shu et al., 2022) and Stein Variational Gradient Descent (Liu & Wang, 2016)) behind a single `Ensemble` abstraction. On top of this, `bensemble` provides tools to decompose predictive uncertainty into aleatoric and epistemic components (Kendall & Gal, 2017), calibrate probabilistic predictions (temperature and vector scaling (Guo et al., 2017)), and evaluate calibration quality (Expected Calibration Error, Brier score, negative log-likelihood). Researchers can train models using ordinary PyTorch code and use `bensemble` for inference-time uncertainty analysis without adopting a new training framework.

Statement of need

Uncertainty quantification (UQ) for deep learning is an active and fragmented research area. Individual methods like Deep Ensembles (Lakshminarayanan et al., 2017), MC Dropout (Gal & Ghahramani, 2016), variational inference with the local reparameterization trick (Kingma et al., 2015), Laplace approximations (Ritter et al., 2018), and Neural Ensemble Search (Zaidi et al., 2021) are each associated with their own reference implementation, typically released as a standalone research artifact tied to a single paper, with inconsistent APIs, inconsistent assumptions about training loops, and little support for combining or comparing methods side-by-side. A researcher who wants to ask a simple applied question "which UQ method gives the best calibrated, best-separated in-distribution vs. out-of-distribution uncertainty for my model, at what compute cost?", currently has to reimplement or glue together several independent codebases, each with different conventions for what a "member," a "sample," or a "posterior" is.

`bensemble` is designed for exactly this audience: researchers and practitioners who already have a working PyTorch training pipeline and want to layer uncertainty estimation, calibration,

and ensembling on top of it without restructuring their code. Because the library treats "where predictions come from" as an implementation detail behind a single `Ensemble.predict_members` interface, switching between, e.g., Deep Ensembles and MC Dropout in an evaluation script requires changing a single constructor call rather than rewriting the evaluation logic.

State of the field

The closest existing work is `torch-uncertainty` (Lafage et al., 2025), a substantially larger and more mature PyTorch-based UQ library covering more methods (including BatchEnsemble, Masksembles, MIMO, and Packed Ensembles) and more tasks (classification, regression, segmentation, depth estimation) than `bensemble`. It does not, however, include Neural Ensemble Search or Probabilistic Backpropagation, and its training/evaluation routines are built around PyTorch Lightning, which requires adopting its routine-based training abstraction rather than layering onto an existing plain-PyTorch training loop. `bensemble`'s design deliberately avoids this: it treats the training loop as the user's own code and only wraps inference-time prediction and uncertainty analysis, at the cost of a narrower method/task scope than `torch-uncertainty`.

Other existing tools address related but narrower parts of the same problem. `laplace-torch` (Daxberger et al., 2021) provides a mature, focused implementation of Laplace approximations for PyTorch models. `Uncertainty Toolbox` (Chung et al., 2021) provides assessment, visualization, and recalibration utilities for regression uncertainty, but is not itself a model-training or ensembling library. NNI (Neural Network Intelligence) (Microsoft, 2021) supports general neural architecture search but has no built-in notion of ensemble diversity or posterior sampling. Relative to this landscape, `bensemble`'s contribution is combining explicit ensembling, implicit/stochastic ensembling, posterior-based sampling, and ensemble-aware architecture search behind one shared abstraction with matching calibration and uncertainty-decomposition utilities, without requiring a Lightning-style training routine, a narrower but more lightweight alternative to `torch-uncertainty` for researchers who want to keep their existing PyTorch training code unchanged.

Software design

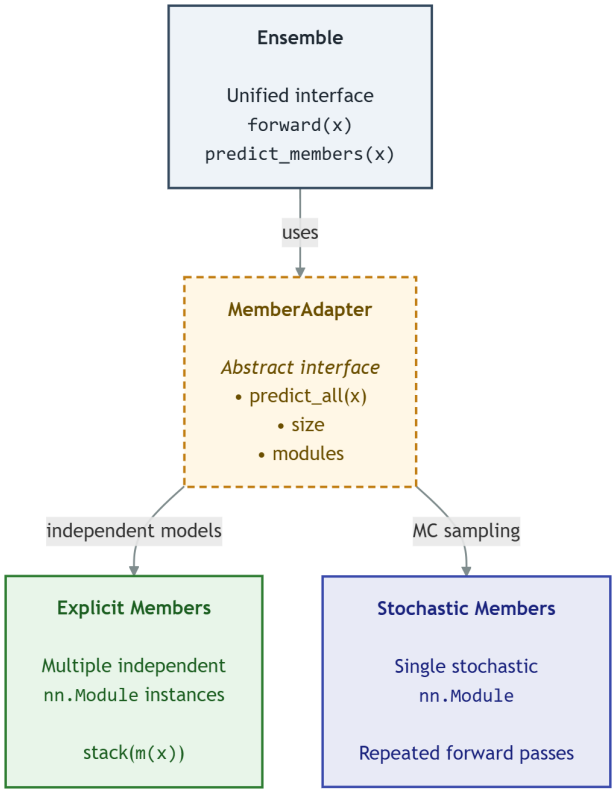

Figure 1: Bensemble architecture

The central design decision in `bensemble` is the separation between an `Ensemble`, a thin, always-present wrapper exposing `predict_members` and a combination rule, and a `MemberAdapter`, which encapsulates how member predictions are produced. Two adapters are provided: `ExplicitMembers`, which wraps a list of independently-parameterized `nn.Module` instances (Deep Ensembles, Laplace/PBP posterior samples, NES-selected ensembles), and `StochasticMembers`, which wraps a single stochastic model and produces multiple samples via repeated forward passes (MC Dropout, variational layers). This split was chosen over a single "list of models" abstraction because several UQ methods, most notably MC Dropout and variational inference, do not have a natural list of independent models at all; forcing them into that shape would require materializing many redundant copies of the same network. The trade-off is that code consuming an `Ensemble` must not assume that `member_modules` always corresponds one-to-one with the number of predictive samples (`num_members`).

Search algorithms (`NESBayesianSampler`, `RandomSearcher`, `EvolutionarySearcher`) are implemented against the same `SearchSpace` protocol and the same `Ensemble.from_models` constructor as the non-search methods, so a searched ensemble is, from the calibration/uncertainty tooling's point of view, indistinguishable from a hand-built Deep Ensemble. This lets users swap in a search-based ensembling strategy without touching any downstream evaluation code. `NESBayesianSampler` operates over a discrete pool of already-trained candidate models rather than a continuous architecture parameterization, so it should be read as a discrete, pool-based heuristic inspired by its namesake rather than a faithful reproduction of it.

For variational Bayesian layers, `bensemble` also provides pruning utilities (`prune_model`, `apply_pruning`) that remove weights whose posterior signal-to-noise ratio (|mean| / standard

deviation) falls below a threshold, following the heuristic proposed by Graves (Graves, 2011), letting users trade some posterior fidelity for a smaller deployed model.

One area for future work is providing more detailed guidance on selecting the `temperature` parameter for posterior-sampling methods such as the Laplace approximation, including additional empirical ablation studies and practical recommendations in the documentation.

Research impact statement

`bensemble` has been actively developed since September 2025 through a rigorous and iterative software engineering process, including versioned PyPI releases, automated testing, documentation, and a pull-request-based workflow.

As a concrete demonstration of correctness and maturity, we validated `bensemble` end-to-end by running eight implemented classification methods, including a deterministic baseline (Single Net), classical UQ methods (Deep Ensembles, MC Dropout, variational inference, and Laplace approximation), and three Neural Ensemble Search variants on a shared image-classification and out-of-distribution-detection setup (CIFAR-10 vs. SVHN), training every method under matched compute budgets. This process surfaced and fixed several subtle correctness issues that are easy to miss in a UQ library, most notably, that our `ExplicitMembers` adapter did not itself manage train/eval mode, causing BatchNorm statistics to leak between ensemble members at inference time, issues we have since covered with regression tests. We view this kind of methodologically careful, cross-method validation as useful for researchers who want to compare UQ methods rather than reimplement one in isolation.

The project also includes automated unit tests and continuous integration to help ensure correctness and compatibility across Python versions 3.10–3.13.

Benchmarks

Beyond unit and integration tests, we validated `bensemble` end-to-end by running eight classification methods on a shared, realistic workload (Single Neural Network baseline, Deep Ensembles, MC Dropout, VI, Laplace, and the three Neural Ensemble Search variants) on CIFAR-10 versus SVHN, and all four regression methods (PBP, VI, Laplace, and a plain MAP baseline) across four UCI datasets, over five random train/test splits per dataset. As a sanity check, rather than a comparison of which method performs best, Table 1 confirms that all eight classification methods completed training successfully and reached a consistent, reasonable accuracy range on CIFAR-10, with no method failing outright or producing degenerate predictions.

Table 1: In-distribution accuracy on CIFAR-10 for all eight classification methods, from a single training run each (see `benchmarks/` for full metrics including calibration and out-of-distribution detection results, and for the regression benchmark).

Method	ID Accuracy
Single Net	92.7%
Deep Ensemble	93.8%
MC Dropout	92.9%
VI	92.9%
Laplace (K-FAC)	92.8%
NESBS (SVGD)	93.7%
NES-RS	93.9%
NES-RE	93.8%

All four regression methods likewise completed successfully across all four UCI datasets; per-dataset RMSE and NLPD for every method, along with a written discussion of methodology

and several caveats we encountered while building this benchmark (matched-budget fairness across search-based and non-search methods, a train/eval-mode handling bug that leaked BatchNorm statistics between ensemble members, and differing rates of noise-hyperparameter recalibration across the regression methods), are reported in the `benchmarks/` directory.

AI usage disclosure

Generative AI tools (Claude by Anthropic) were used during two phases of this project. First, during early method prototyping, AI assistance was used to help develop initial prototypes of some methods. These implementations were subsequently reviewed, refactored, and rewritten by the authors over the following weeks. No AI-assisted code was merged without a human author verifying it against the relevant published method description and against automated tests. Second, AI assistance was used during preparation of the cross-method benchmark described in the Research impact statement, specifically to help identify methodological issues and to help draft this manuscript. All AI-assisted code changes were manually reviewed, tested against the library's existing test suite, and checked against the relevant library source code before being committed.

Acknowledgements

The authors received no external funding for this work.

References

Chung, Y., Char, I., Guo, H., Schneider, J., & Neiswanger, W. (2021). Uncertainty toolbox: An open-source library for assessing, visualizing, and improving uncertainty quantification. *arXiv Preprint arXiv:2109.10254*.

Daxberger, E., Kristiadi, A., Immer, A., Eschenhagen, R., Bauer, M., & Hennig, P. (2021). Laplace redux – effortless Bayesian deep learning. *Advances in Neural Information Processing Systems*, *34*.

Gal, Y., & Ghahramani, Z. (2016). Dropout as a bayesian approximation: Representing model uncertainty in deep learning. *International Conference on Machine Learning*, 1050–1059.

Graves, A. (2011). Practical variational inference for neural networks. *Advances in Neural Information Processing Systems*, *24*, 2348–2356.

Guo, C., Pleiss, G., Sun, Y., & Weinberger, K. Q. (2017). On calibration of modern neural networks. *International Conference on Machine Learning*, 1321–1330.

Hernández-Lobato, J. M., & Adams, R. (2015). Probabilistic backpropagation for scalable learning of Bayesian neural networks. *International Conference on Machine Learning*, 1861–1869.

Kendall, A., & Gal, Y. (2017). What uncertainties do we need in Bayesian deep learning for computer vision? *Advances in Neural Information Processing Systems*, *30*.

Kingma, D. P., Salimans, T., & Welling, M. (2015). Variational dropout and the local reparameterization trick. *Advances in Neural Information Processing Systems*, *28*.

Lafage, A., Laurent, O., Gabetni, F., & Franchi, G. (2025). Torch-uncertainty: A deep learning framework for uncertainty quantification. *Advances in Neural Information Processing Systems 38 (NeurIPS 2025), Datasets and Benchmarks Track*.

Lakshminarayanan, B., Pritzel, A., & Blundell, C. (2017). Simple and scalable predictive uncertainty estimation using deep ensembles. *Advances in Neural Information Processing*

Systems, *30*.

Li, Y., & Turner, R. E. (2016). Rényi divergence variational inference. *Advances in Neural Information Processing Systems*, *29*, 1073–1081.

Liu, Q., & Wang, D. (2016). Stein variational gradient descent: A general purpose Bayesian inference algorithm. *Advances in Neural Information Processing Systems*, *29*.

Microsoft. (2021). *NNI (neural network intelligence)*. https://github.com/microsoft/nni

Ritter, H., Botev, A., & Barber, D. (2018). A scalable laplace approximation for neural networks. *International Conference on Learning Representations*.

Shu, Y., Chen, Y., Dai, Z., & Low, B. K. H. (2022). Neural ensemble search via Bayesian sampling. *Proceedings of the Thirty-Eighth Conference on Uncertainty in Artificial Intelligence*, 1803–1812.

Zaidi, S., Zela, A., Elsken, T., Holmes, C. C., Hutter, F., & Teh, Y. W. (2021). Neural ensemble search for uncertainty estimation and dataset shift. *Advances in Neural Information Processing Systems*, *34*.